

计算机操作系统

倪福川

fcni_cn@mail.hzau.edu.cn

华中农业大学 信息学院

目 录

[第一章 操作系统引论](#)

[第二章 进程的描述与控制](#)

[第三章 处理机调度与死锁](#)

[第四章 存储器管理](#)

[第五章 虚拟存储器](#)

[第六章 输入输出系统](#)

[第七章 文件管理](#)

[第八章 磁盘存储器的管理](#)

[第九章 操作系统接口](#)

[第十二章 保护和安全](#)

第四章 存储器管理

4.1 存储器的层次结构

4.2 程序的装入和链接

4.3 连续分配存储管理方式

4.4 对换(Swapping)

4.5 分页存储管理方式

4.6 分段存储管理方式

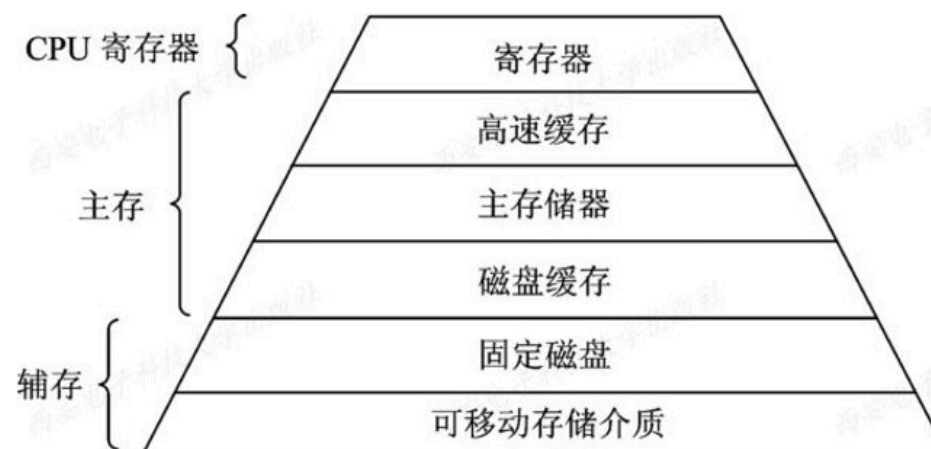
4.1 存储器的层次结构

指令执行涉及对存储器访问:

- 速度 能与处理机速度相匹配;
- 容量 非常大的;
- 价格 很便宜。

4.1.1 多层结构的存储器系统

1. 存储器的多层结构



• 图4-1 计算机系统存储层次示意

2. 可执行存储器

寄存器和主存储器访问:

进程在很少的时钟周期内 用load或store指令访问。

访问机制，所耗费时间不同

辅存访问：通过I/O设备实现

涉及中断、设备驱动程序以及物理设备运行，所耗费时间远远高于访问可执行存储器的时间，相差3个数量级以上。

4.1.2 主存储器与寄存器

1. 主存储器

可执行存储器

简称内存或主存，主要部件，用于保存进程运行时的程序和数据。

2. 寄存器

与处理机相同的速度，访问速度最快，能与CPU协调工作，价格十分昂贵，容量不可能很大。

4.1.3 高速缓存和磁盘缓存

1. 高速缓存

介于寄存器和存储器之间的存储器，主要用于备份主存中较常用的数据，可大幅度地提高程序执行速度。

容量远大于寄存器，比内存约小，
访问速度快于主存储器。

2. 磁盘缓存

磁盘I/O速度远低于对主存访问速度，为缓和两者速度不匹配，设置磁盘缓存。

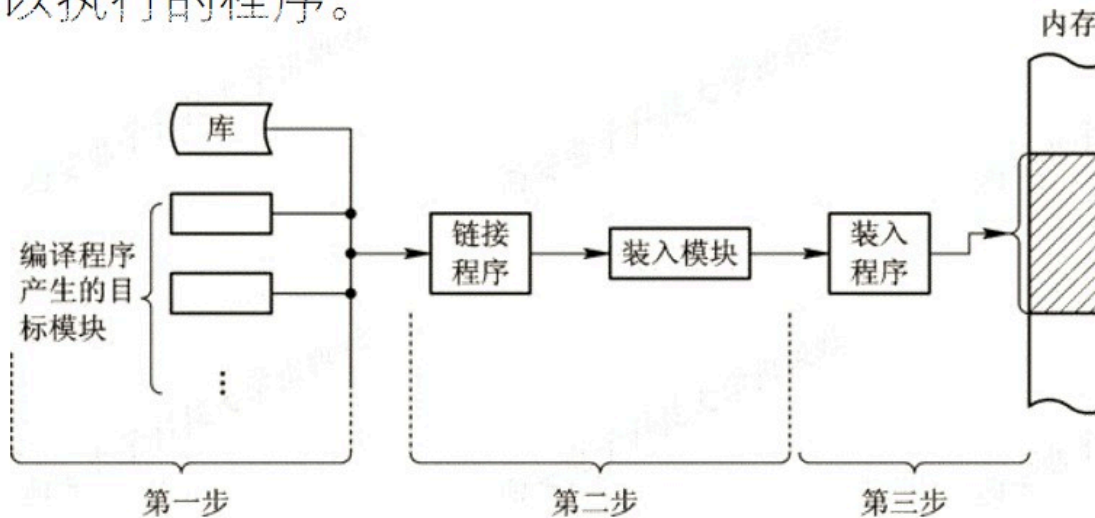
主要暂存放频繁使用部分磁盘数据和信息，减少访问磁盘次数。

本身并不是实际存在存储器，利用主存中部分存储空间暂时存放从磁盘中读出(或写入)信息。

主存也可以看作是辅存的高速缓存

4.2 程序的装入和链接

用户程序，经过编译、连接，装入内存，将其转变为一个可以执行的程序。



• 图4-2 对用户程序的处理步骤

- **内存空间**（或物理空间、绝对空间）
 - 由**内存**一系列存储单元所限定的地址范围

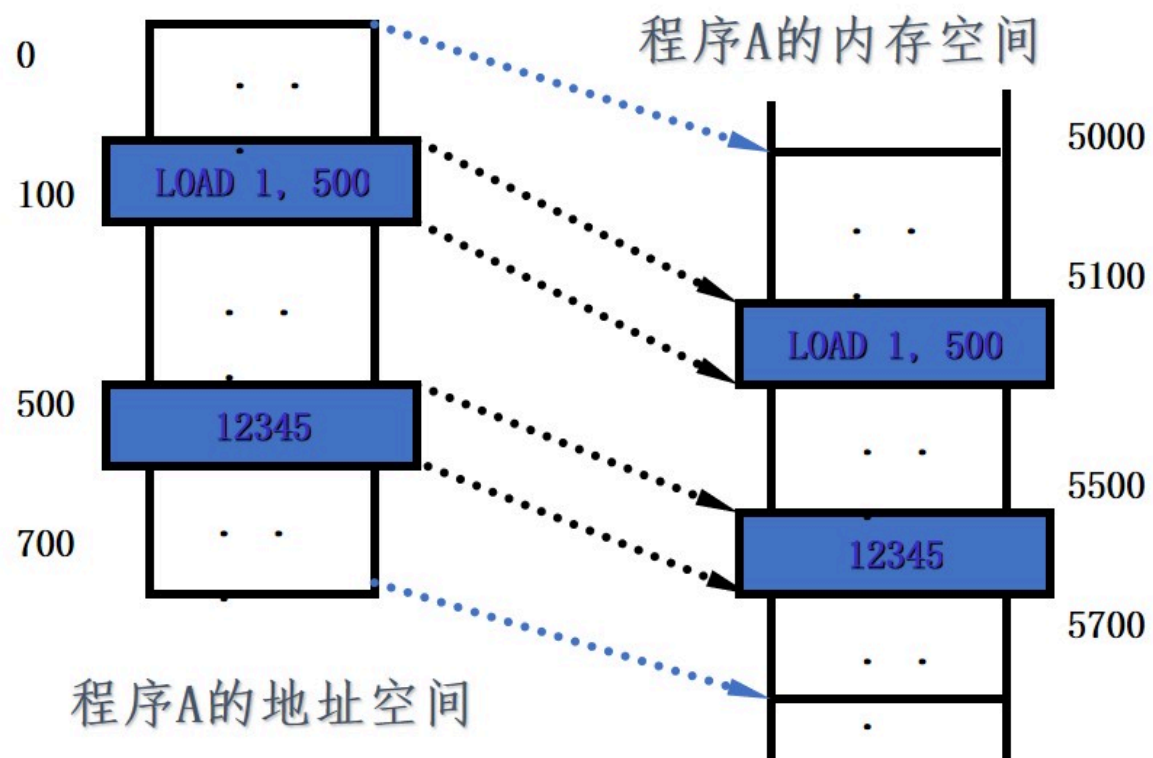
逻辑地址空间（或地址空间）

由**程序**中逻辑地址组成的地址范围

相对地址（或逻辑地址） 程序经编译后的每个目标模块都以0为基地址顺序编址

相对地址

绝对地址



4.2.1 程序的装入

- 1. 绝对装入方式 (Absolute Loading Mode)
- 2. 可重定位装入方式 (Relocation Loading Mode)
- 3. 动态运行时装入方式 (Dynamic Run-time Loading)

4.2.1 程序的装入

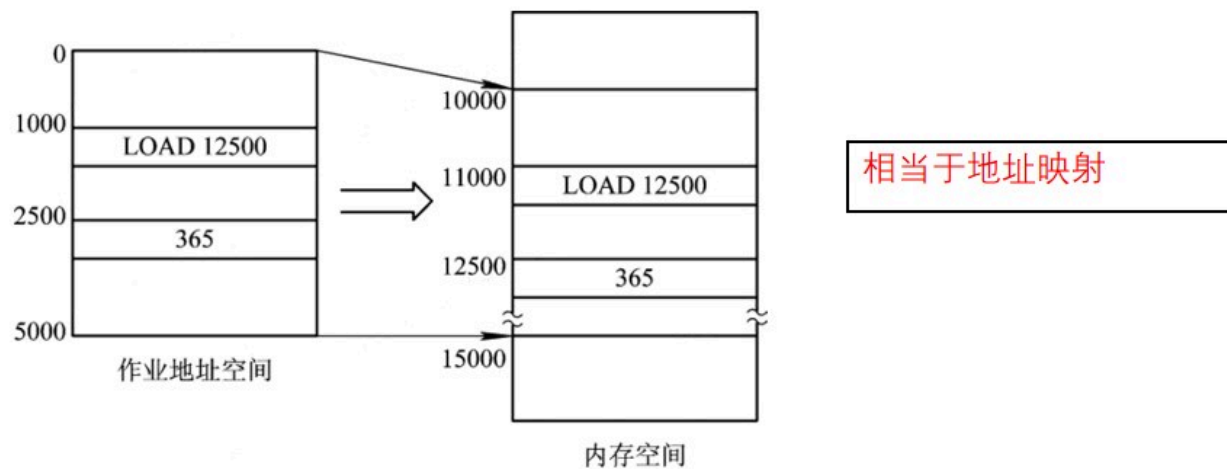
1. 绝对装入方式 (Absolute Loading Mode)

系统很小，仅运行单道程序，程序驻留在内存指定位置。
程序经编译，产生绝对地址(即物理地址)的目标代码。

2. 可重定位装入方式 (Relocation Loading Mode)

用户程序编译形成目标模块，起始地址都是从0开始的，程序其它地址也相对于起始地址计算。

重定位：装入时对目标程序中指令和数据的修改过程。



3. 动态运行时装入方式 (Dynamic Run-time Loading)

将装入模块装入到内存中任何允许的位置； 程序运行时允许在内存中移动位置。

多道程序环境

边运行，边装入，动态的；
重定位开始时一次性装入，在运行时不再改变

在程序执行时将**相对地址**转换成为**绝对地址**

4.2.2 程序的链接

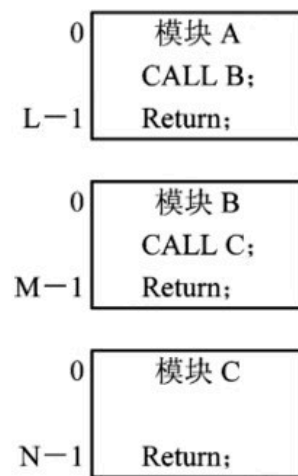
- 1. 静态链接 (Static Linking) 方式
- 2. 装入时动态链接 (Load-time Dynamic Linking)
- 3. 运行时动态链接 (Run-time Dynamic Linking)

4.2.2 程序的链接

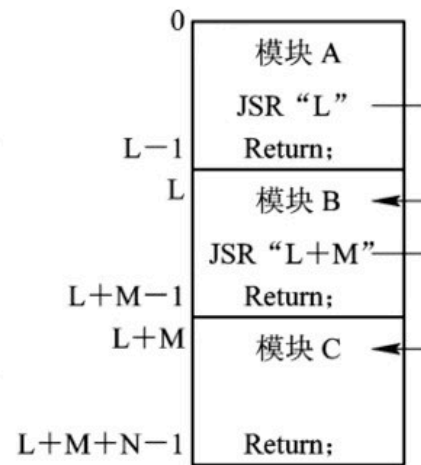
1. 静态链接(Static Linking)方式

各目标模块及所需库函数链接成装配模块，须：

- 1) 对相对地址进行修改
- 2) 变换外部调用符号。



(a) 目标模块



(b) 装入模块

2. 装入时动态链接 (Load-time Dynamic Linking)

目标模块，边装入边链接。

装入时，若发生外部模块调用事件，装入程序去找出相应外部目标模块，并将它装入内存，优点：

- (1) 便于修改和更新。
- (2) 便于实现对目标模块的共享。

3. 运行时动态链接 (Run-time Dynamic Linking)

将某些目标模块的链接，推迟到**执行**时才进行。

在**执行过程**中，若发现一个被调用模块尚未装入内存时，由操作系统去找到该模块，将它装入内存，并把它**链接**到调用者模块上。

4.3 连续分配存储管理方式

用户程序分配连续的内存空间;

程序或数据的逻辑地址相邻, 在内存空间分配的物理地址相邻

- 1 单一连续分配
- 2 固定分区分配
- 3 动态分区分配
- 4 动态可重定位分区分配

4.3 连续分配存储管理方式

4.3.1 单一连续分配

单道程序环境

内存:系统区和用户区

{ 系统区, 仅给OS用,
放在内存低址部分。
用户区, 仅一道程序,
独占整个用户空间。



单一连续分配

4.3.2 固定分区分配

1. 划分分区的方法

(1) 分区大小相等

(所有内存分区大小相等)。

(2) 分区大小不等。



固定分区分配
(分区相等)



固定分区分配
(分区不等)

2. 内存分配

分区按大小排队，建立分区使用表，表项包括分区起始地址、大小及状态(是否分配)

分区号	大小(KB)	起址(K)	状态
1	12	20	已分配
2	32	32	已分配
3	64	64	未分配
4	128	128	已分配

(a) 分区说明表

空间	操作系统
24 KB	作业 A
32 KB	作业 B
64 KB	作业 C
128 KB	
...	...
256 KB	

(b) 存储空间分配情况

- 图4-5 固定分区使用表

4.3.3 动态分区分配

1. 动态分区分配中的数据结构

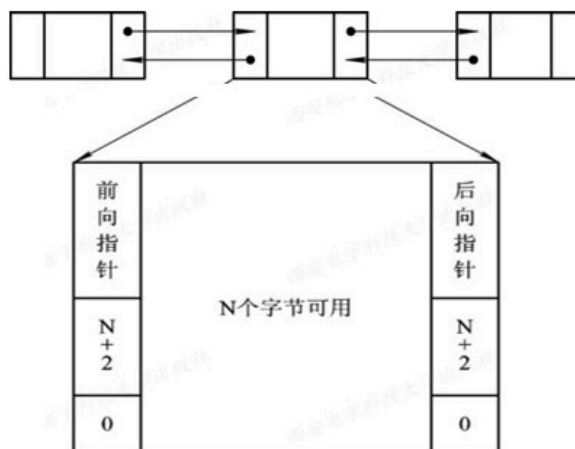
① 空闲分区表，记录空闲分区情况。表目包括分区号、分区大小和分区始址等数据项

分区号	分区大小(KB)	分区始址(K)	状态
1	50	85	空闲
2	32	155	空闲
3	70	275	空闲
4	60	532	空闲
5	...	...	...

• 图4-6 空闲分区表

② 空闲分区链,

分区的起始部分: 设置用于控制分区的信息、向前指针,
分区尾部: 设置一个向后指针, 形成双向链表。



• 图4-7 空闲链结构

2. 动态分区分配算法

新作业装入，按算法分配内存，从空闲分区表或空闲分区链中选出一分区分配给该作业。

顺序搜索的动态分区分配算法

- 首次适应(first fit, FF)算法
- 循环首次适应(next fit, NF)算法
- 最佳适应(best fit, BF)算法
- 最坏适应(worst fit, WF)算法

动态分区分配算法：

基于索引搜索的动态分区分配算法

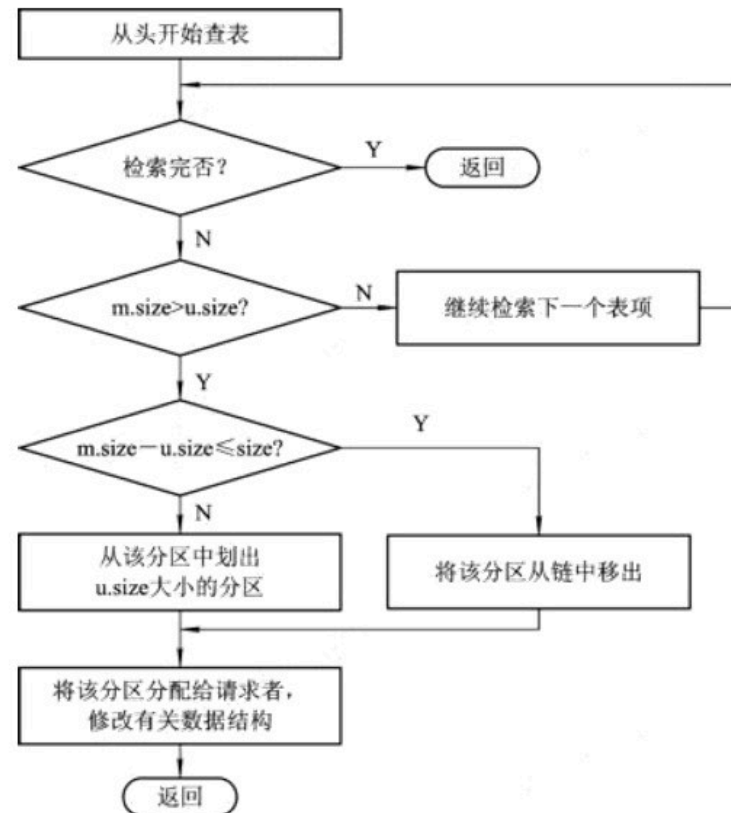
- 1. 快速适应(quick fit)算法
- 2. 伙伴系统(buddy system)
- 3. 哈希算法

3. 分区分配操作

1) 分配内存

利用某种分配算法，从空闲分区链(表)中找到所需大小的分区。

设请求分区大小为 $u.size$ ，
表中空闲分区的大小为 $m.size$ 。

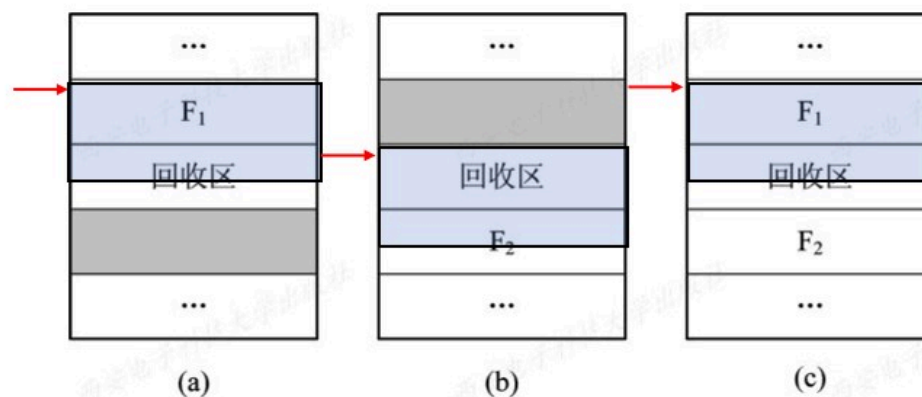


• 图4-8 内存分配流程

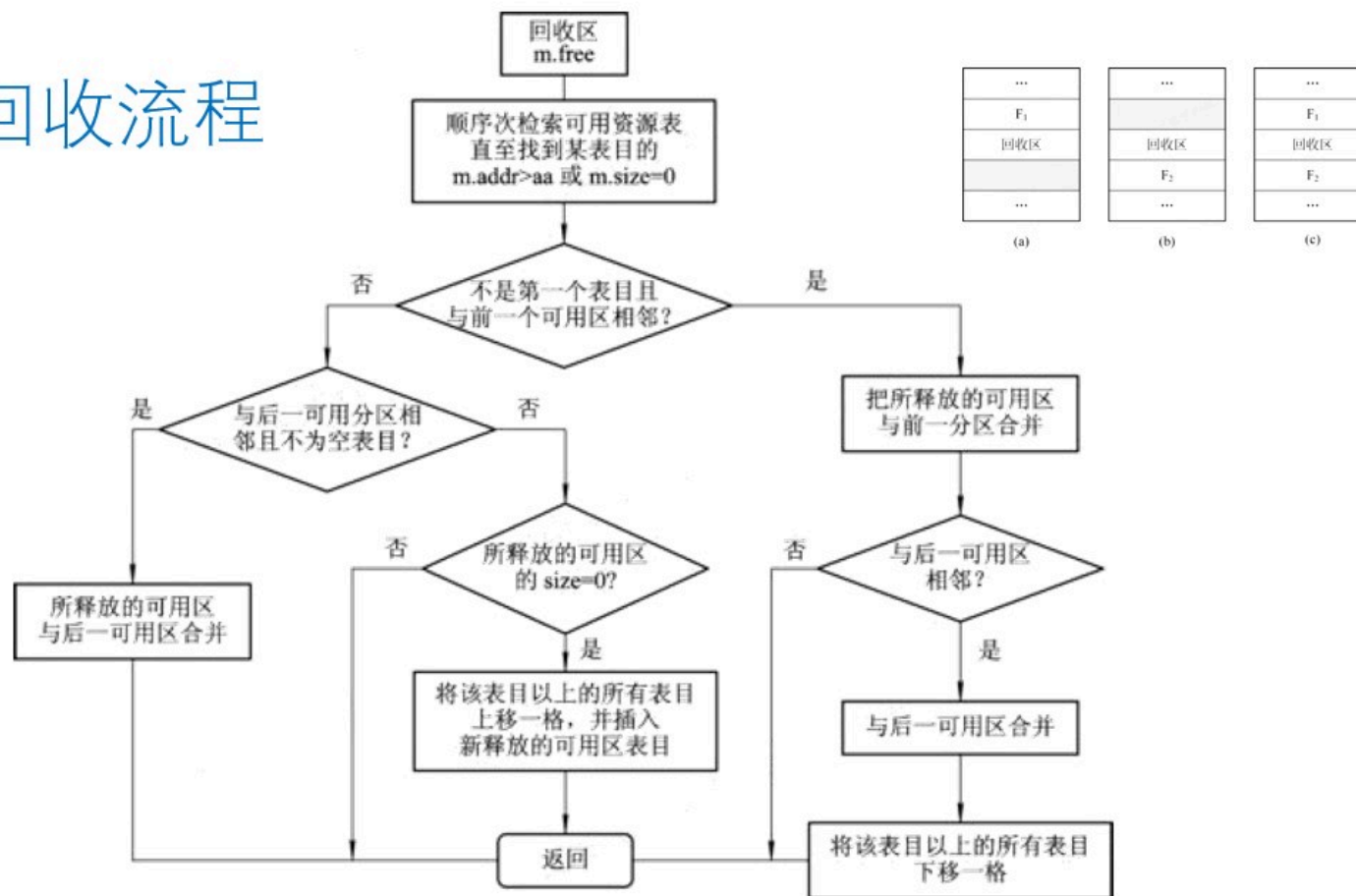
2) 回收内存

根据回收区首址，从空闲区链(表)中找相应的插入点：

- (1) 回收区与插入点的前一个空闲分区 F_1 相邻接 (a)
- (2) 回收分区与插入点的后一空闲分区 F_2 相邻接 (b)
- (3) 回收区同时与插入点的前、后两个分区邻接 (c)
- (4) 回收区既不与 F_1 邻接，又不与 F_2 邻接。



内存回收流程



• 图4-10 内存回收流程

4.3.4 基于顺序搜索的动态分区分配算法

- 1. 首次适应(first fit, FF)算法
- 2. 循环首次适应(next fit, NF)算法
- 3. 最佳适应(best fit, BF)算法
- 4. 最坏适应(worst fit, WF)算法

1. 首次适应(first fit, FF)算法

空闲分区 地址递增次序链接。

分配时，从链首顺序查找，直至找到一个大小能满足要求的空闲分区为止。

再按照作业的大小，从该分区划出一块内存空间，分配给请求者，余下空闲分区仍留在空闲链中。

若从链首直至链尾都不能找到一个能满足要求的分区，内存分配失败，返回。

系统中已没有足够大的内存分配给该进程

2. 循环首次适应(next fit, NF)算法

从上次找到的空闲分区的下一个空闲分区开始查找，直至找到一个能满足要求的空闲分区，从中划出一块与请求大小相等的内存空间分配给作业。

避免低址部分留下许多很小的空闲分区，减少查找可用空闲分区的开销，

3. 最佳适应(best fit, BF)算法

每次为作业分配内存时，总是把能满足要求、又是最小的空闲分区分配给作业。

要求空闲分区按其容量以从小到大的顺序形成一空闲分区链。

4. 最坏适应(worst fit, WF)算法

扫描整个空闲分区表或链表时，总是挑选一个最大的空闲区，从中分割一部分存储空间给作业使用

最坏适应分配算法策略好与最佳适应算法相反：导致存储器中缺乏大的空闲分区

4.3.5 基于索引搜索的动态分区分配算法

- 
1. 快速适应 (quick fit) 算法
 2. 伙伴系统 (buddy system)
 3. 哈希算法

1. 快速适应(quick fit)算法

根据空闲分区容量分类，对相同容量空闲分区设立一空闲分区链表；同时设立管理索引表，每个索引表项对应一种空闲分区类，记录该类空闲分区链表的表头指针。

搜索空闲分区时，第一步根据进程的长度，从索引表中寻找能容纳它的最小空闲区链表；第二步是从链表中取下第一块进行分配即可。

2. 伙伴系统(buddy system)

用户提出申请时，分配一块大小“恰当”的内存区给用户；反之在用户释放内存区时即回收。

占用块或空闲块，其大小均为 2^k （ k 为正整数）

开始时，整个内存区大小为 2^m 的空闲分区。

在运行中，不断划分，形成若干个不连续的空闲分区，按分区的大小进行分类。

具有相同大小的所有空闲分区，单独设立一个空闲分区双向链表

2、伙伴系统:

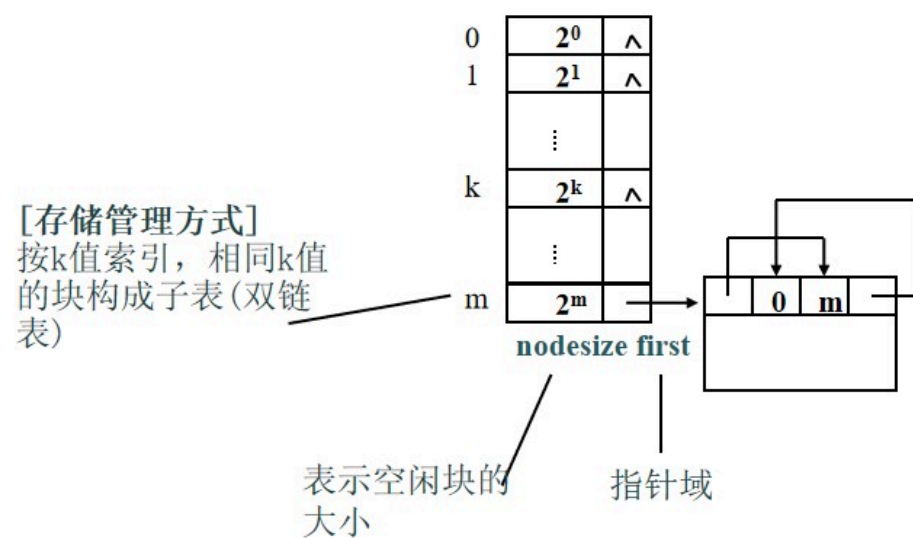
分配时，设需分配长度为 n ，找 2^i 分区链的分区，使 $2^{i-1} < n < 2^i$ 若无，找 2^{i+1} 且把它均分两块，称为伙伴。一个加入 2^i 分区链，一个分配；

回收时，若已存在 2^i 空闲分区，则将其于伙伴合并为 2^{i+1} 分区，

- **特点**：性能取决于查找空闲分区的位置和分割、合并的时间。时间上不及快速适应算法，但空闲分区的使用率高

2. 伙伴系统 (buddy system)

- 例：初始状态如下图所示，其中 m 个子表都为空表，只有大小为 2^m 的链表中有一个结点，即整个存储空间。



回收算法

考虑互为伙伴的空闲区的归并

在伙伴系统中，对于一个大小为 2^k ，地址为 x 的内存块，其伙伴块的地址则用 $buddy_k(x)$ 表示，其通式为：

$$buddy_k(x) = \begin{cases} x + 2^k & (\text{若 } x \bmod 2^{k+1} = 0) \\ x - 2^k & (\text{若 } x \bmod 2^{k+1} = 2^k) \end{cases}$$



3、哈希算法：

建立哈希函数，构造一张一空闲分区大小为关键字的哈希表，该表的每一个表项记录一个对应的空闲分区链表。

分配时，根据所需空闲分区大小，通过哈希函数计算，即得到在哈希表中的位置，再分配

利用哈希快速查找优点，以及空闲分区在可利用空闲区表中的分布规律，

4.3.6、可重定位分区分配

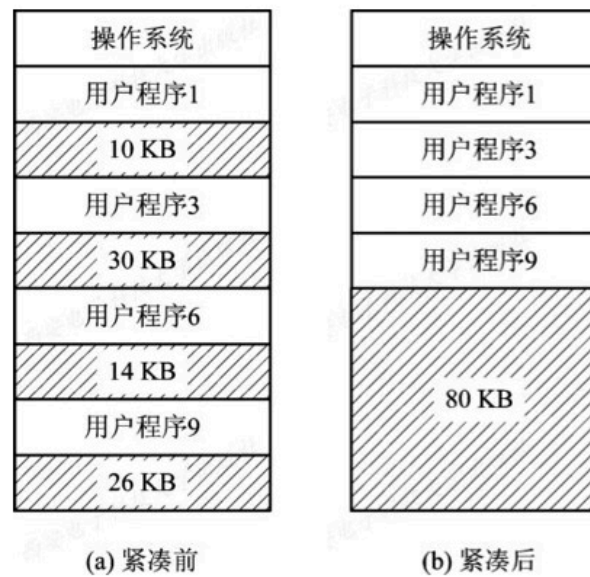
- **1、动态重定位的引入**

- 随系统接收作业增加，内存中连续大块分区不复存在，产生了大量的“**碎片**”。
- 新作业无法装入到“碎片”小分区上运行，但所有碎片总和可能大于需求。
- 通过“**拼接**”或“**紧凑**”来实现程序的浮动（**动态重定位**）。

紧凑

碎片

外部碎片
内部碎片



• 图4-11 紧凑的示意图

2、动态重定位的实现

须由硬件地址变换机构支持

重定位寄存器：存放程序在内存中的起始地址。

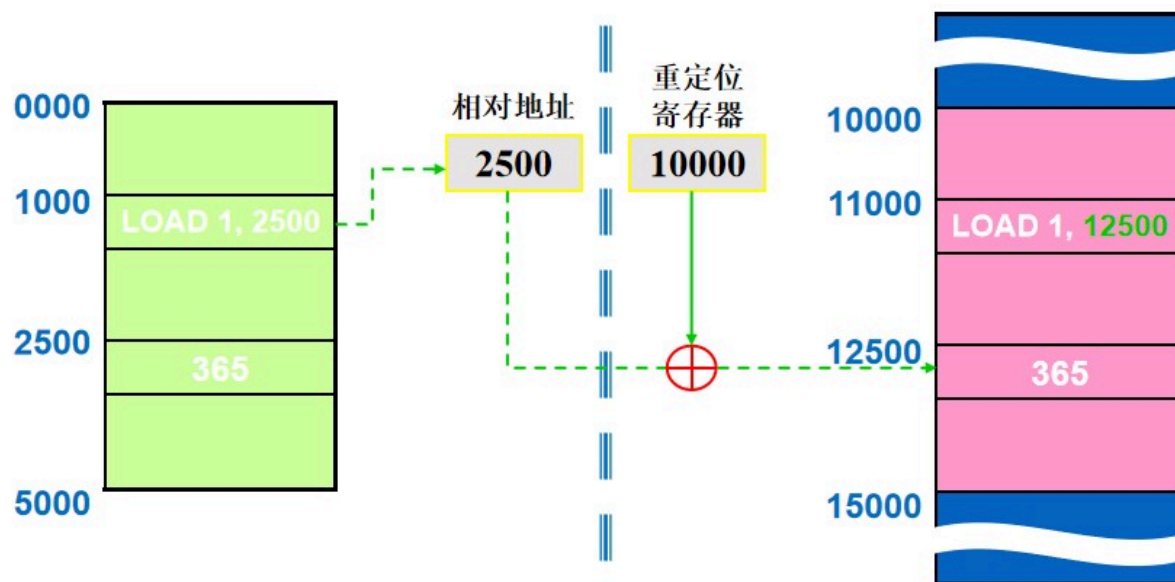


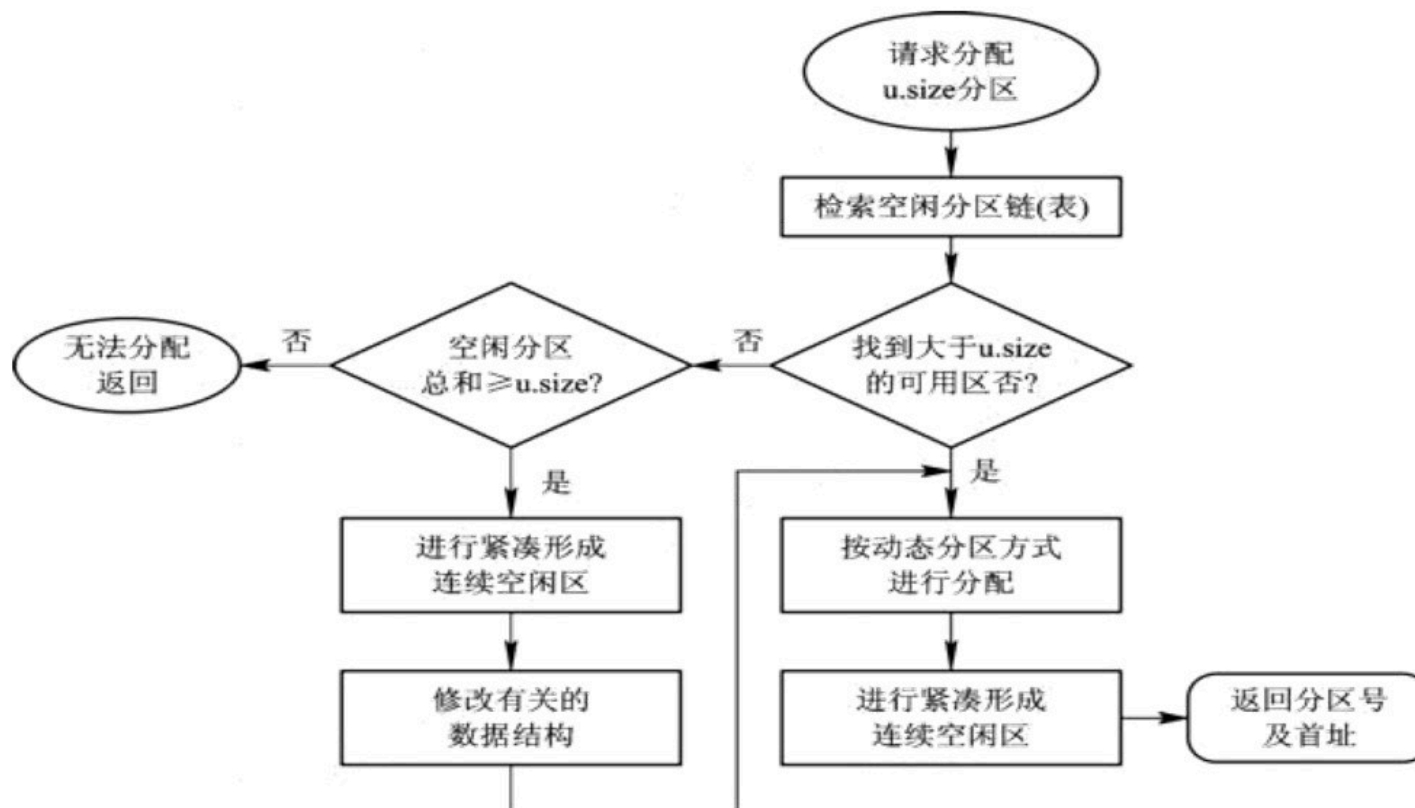
图4-12 动态重定位示意图

3. 动态重定位分区分配算法

动态重定位分区分配算法与动态分区分配算法基本上相同，差别仅在于：在这种分配算法中，增加了**紧凑**的功能。

当该算法不能找到一个足够大的空闲分区以满足用户需求时，如果所有的小的空闲分区的容量总和大于用户的要求，这时便须对内存进行“紧凑”，将经“紧凑”后所得到的大空闲分区分配给用户。

如果所有的小的空闲分区的容量总和仍小于用户的要求，则返回分配失败信息。



• 图4-13 动态分区分配算法流程图

4.4 对换(Swapping)

也称为交换技术，系统把所有的用户作业存放在磁盘上，每次只能调入一个作业进入内存，当该作业的一个时间片用完时，将它调至外存的后备队列上等待，再从后备队列上将另一个作业调入内存。

最早用于MIT的单用户分时系统CTSS中。当时计算机的内存都非常小，为了能分时运行多个用户程序而引入了对换技术。

4.4.1 多道程序环境下的对换技术

1. 对换的引入

在多道程序环境下，被阻塞进程占用大量的内存空间，可能出现在内存中所有进程都被阻塞，而无可运行之进程，迫使CPU停止下来等待的情况；另一方面，却又有着许多作业，因内存空间不足，一直驻留在外存上，而不能进入内存运行。

系统资源浪费严重，系统吞吐量下降。

2. 对换的类型

在每次对换时，都是将一定数量的程序或数据换入或换出内存。

根据每次对换时所对换的数量，分为两类：

- (1) 整体对换。
- (2) 页面(分段)对换。

4.4.2 对换空间的管理

1. 对换空间管理的主要目标

在具有对换功能的OS中，通常把磁盘空间分为文件区和对换区两部分。

1) 对文件区管理的主要目标

文件存储空间利用率 离散

2) 对对换空间管理的主要目标

换入换出速度 连续

2. 对换区空闲盘块管理中的数据结构

记录外存对换区中的空闲盘块的使用情况。

其数据结构的形式与内存在动态分区分配方式中所用数据结构相似，即同样可以用空闲分区表或空闲分区链。

在空闲分区表的每个表目中，包含两项：对换区的首址及其大小，分别用盘块号和盘块数表示。

3. 对换空间的分配与回收

连续分配方式，与动态分区方式时的内存分配与回收方法雷同。

其分配算法：可以是首次适应算法、循环首次适应算法或最佳适应算法等。

具体的分配操作也与内存的分配过程相同。

4.4.3 进程的换出与换入

1. 进程的换出

对换进程在实现进程换出时，是将内存中的某些进程调出至对换区，以便腾出内存空间。换出过程可分为以下两步：

- (1) 选择被换出的进程。
- (2) 进程换出过程。

2. 进程的换入

对换进程将定时执行换入操作，它首先查看PCB集合中所有进程的状态，从中找出“就绪”状态但已换出的进程。

当有许多这样的进程时，它将选择其中**已换出到磁盘上时间最久** (必须大于规定时间，如2s) 的进程作为换入进程，为它申请内存。

如果申请成功，可直接将进程从外存调入内存；如果失败，则需先将内存中的某些进程换出，腾出足够的内存空间后，再将进程调入。

主观题 10分

在本课堂上，你想学习那些内容？
你有那些建议？



投票(匿名) 最多可选5项

在线听课评价:

- ☐ A 教师准备充分,讲述条理清晰
- ☐ B 能调动学习的积极性
- ☐ C 教学方式单一、缺乏互动
- ☐ D 学生的收获少,获得感不强
- ☐ E 内容枯燥、听不懂

